



TrainInBlue

PRODUCT DOCUMENTATION

TrainInBlue

Build. Train. Resume.

An offline-first Flutter app for building, organizing, completing, and safely resuming a single workout session. No backend, no account, no network — every change is saved on the device and restored exactly where you left off.

• Flutter & Dart

• Offline-first

• Cubit architecture

• Material 3

Inside this document

A guided tour of the product — from brand and design system to screens, architecture, and how to run it.

01	Product overview	03
	What TrainInBlue is and who it's for	
02	Design system	04
	Color palette, typography, surfaces	
03	Screens & flow	05
	Splash, onboarding, home, detail, add exercise	
04	Architecture	08
	Layered structure & folder map	
05	State & persistence	09
	Cubits, versioned snapshot, recovery	
06	Tech stack	10
	Dependencies & the reason for each	
07	Testing & quality	11
	Unit, widget, and verification approach	
08	Getting started	12
	Run from a clean checkout	

One session, arranged your way

TrainInBlue ships with six starter exercises from bundled JSON. It saves every change on the device and restores an unfinished session exactly after the app is closed and reopened. Tap any exercise for a full detail experience — its own photo, key numbers, an interactive set tracker or a countdown timer, coaching content, and complete / edit / delete actions.

6

Starter exercises

0

Backend services

100%

Offline-first

38

Passing tests

What makes it different



Resumes exactly

Order and completion are restored precisely after the app is closed — an unfinished session is ready when you return.



Safe by design

Every mutation is persisted atomically as one document. Corrupt data is never silently overwritten; you get an explicit recovery choice.



Two target types

Rep-based exercises get an interactive set tracker; duration exercises get a foreground countdown timer with its own ring.



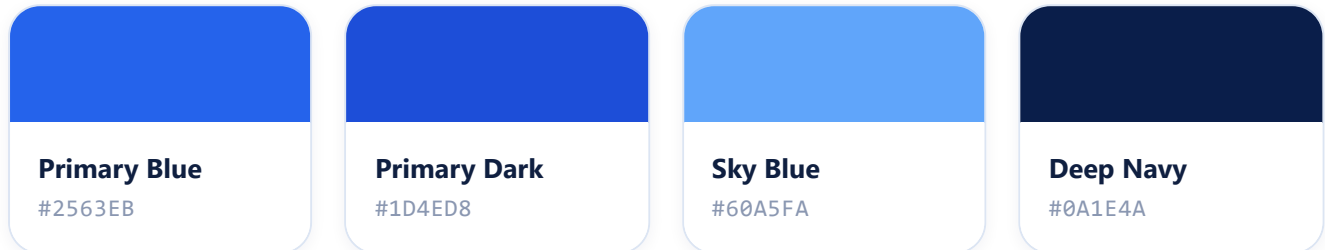
One consistent brand

Deep navy + cobalt `#2563EB` across every photo, icon and surface, so the app feels like one product.

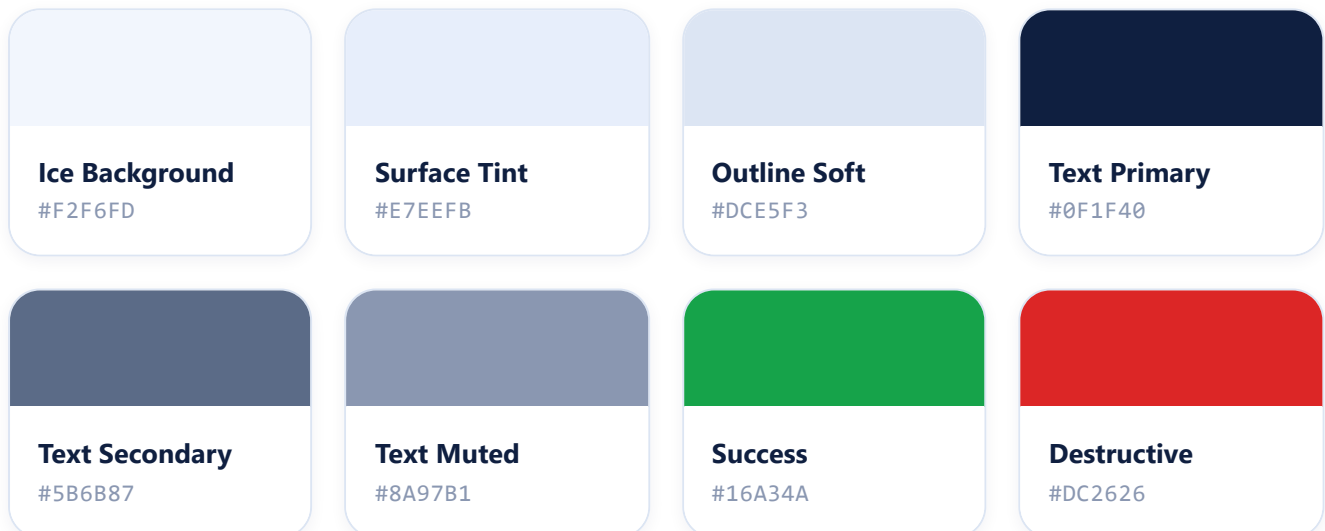
The TrainInBlue palette

Every color in the app comes from a single palette (`AppColorPalette`) so the blue identity stays consistent and easy to adjust from one place.

Brand blues



Surfaces & text



Typography — Manrope

Aa

Weights 400–800 are bundled. Headlines use ExtraBold with tight tracking; body copy uses Regular/Medium for calm, legible reading.

Regular 400

Medium 500

Semi 600

Extra 800

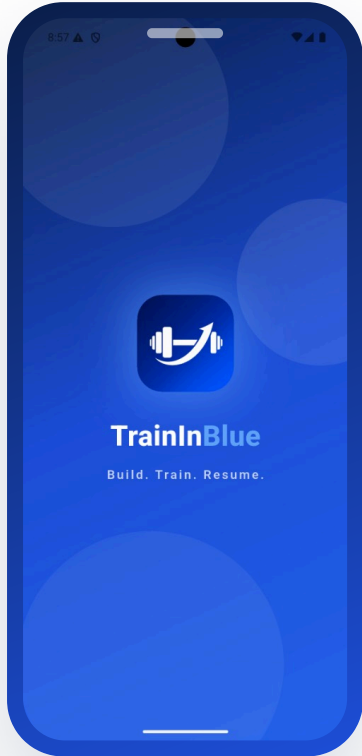
Gradients & depth



The brand gradient (navy → cobalt → sky) powers heroes, primary actions and progress. Soft navy-tinted shadows give cards a premium, floating feel.

From launch to first rep

A calm, photographic onboarding sets the tone before dropping you straight into your single, resumable session.



◆ Splash

A brand-first launch

The animated splash centers the TrainInBlue mark over the signature navy gradient with soft floating orbs, then routes to onboarding or straight to the workout depending on saved state.

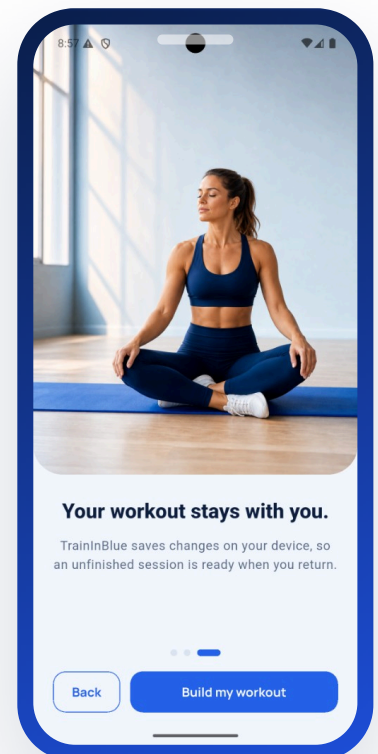
- ☐ Deep navy → cobalt gradient background
- ☐ Brand mark with the "Build. Train. Resume." tagline
- ☐ Decides the entry route from persisted flags

● Onboarding

Three photographic promises

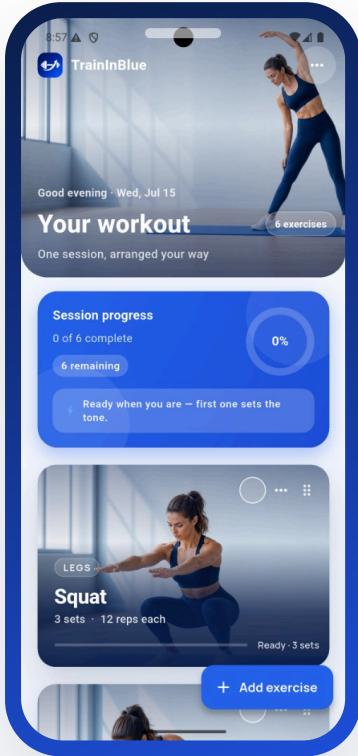
A short, swipeable intro explains the value in three frames — build, track progress, and resume. The final page confirms that your work stays on the device.

- ☐ Next / Back / Skip with a persisted completion flag
- ☐ Dot page indicator and a clear final call-to-action
- ☐ Consistent studio art direction across all photos



Your session, at a glance

The home screen is the single source of truth for the active workout — live progress up top, reorderable exercise cards below.



Workout home

Progress you can feel

A photographic hero greets you by daypart and date. The progress card derives completed / total / remaining / percent in exactly one place, so every surface agrees.

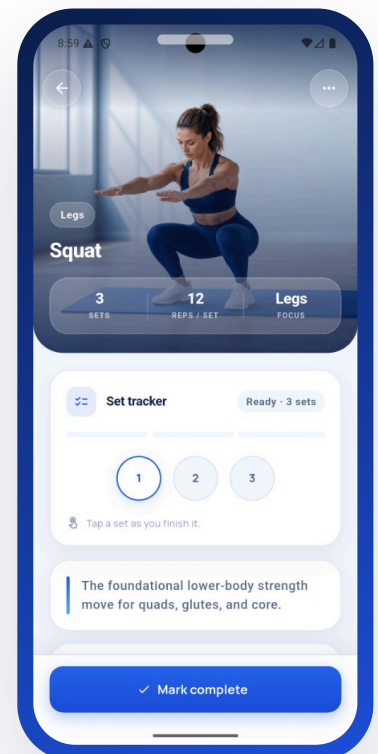
- ☐ Live session progress ring and “remaining” count
- ☐ Rich exercise cards with category, sets and reps
- ☐ Drag handle to reorder · “Add exercise” quick action
- ☐ Safe empty state renders a calm 0%

✓ Exercise detail

Coaching for every move

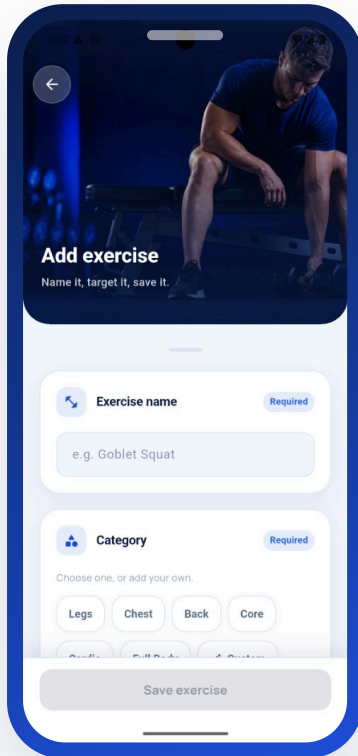
Each exercise opens to its own photo and key numbers. Rep exercises show a tap-to-complete set tracker; duration exercises show a foreground countdown. Coaching steps, tips and equipment sit below.

- ☐ Interactive set tracker — tap a set as you finish it
- ☐ Hero stat strip: sets · reps/set · focus
- ☐ Mark complete, edit, or delete from one place



Add anything, in seconds

A focused form makes creating an exercise effortless — name it, pick a category (or add your own), choose a target, save it.



+ Add exercise

Name it, target it, save it

A photographic header frames the task, then clearly-sectioned cards guide input. Required fields are badged, and the save bar stays disabled until the form is valid.

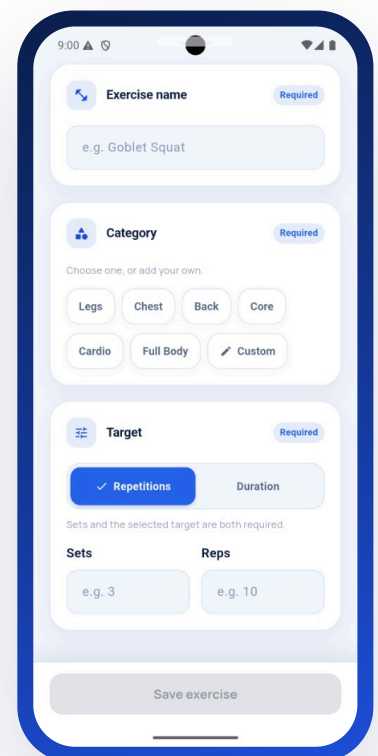
- ☐ Validated exercise name with inline hints
- ☐ Quick-pick categories plus a custom option
- ☐ Repetitions ↔ Duration target toggle

⚙️ Target & category

Structured, forgiving input

Categories are free text with quick-pick suggestions. Sets and the selected target are both required, so saved data is always complete and renderable.

- ☐ Legs · Chest · Back · Core · Cardio · Full Body · Custom
- ☐ Sets + reps for rep exercises; sets + duration otherwise
- ☐ Known exercises get a photo; others fall back gracefully



One responsibility per layer

Screens render state and dispatch intent, the cubit owns rules and persistence ordering, and repositories own storage details. Each concern lives in exactly one place.

UI — screens / components / widgets

Render state · dispatch intent

PRESENTATION



WorkoutCubit · OnboardingCubit · ThemeCubit

Load · mutate · persist · derive progress

STATE (BLOCS/)



WorkoutRepository · OnboardingRepository

Own storage details behind a small API

DATA



LocalStorageService (SharedPreferences)

Source of truth · assets/data/exercises.json seeds first launch only

STORAGE

Folder structure

```
lib/
  Blocs/           # Cubits: workout session, onboarding flag, theme
  screens/         # splash, onboarding, workout home, detail, form
  components/      # reusable cards, buttons, Inputs, dialogs, progress
  widgets/         # small shared widgets (status view, page dots)
  Core/
    constants/     # colors, strings, asset paths, widget keys
    services/      # LocalStorageService, NavigationService
    Utils/         # validators, duration formatter, id generator
    errors/        # readable application exceptions
  data/
    models/        # Exercise, drafts, snapshot, progress
    repositories/  # local workout + onboarding repositories
  app_theme/       # Material 3 theme, typography, color extension
```


Predictable state, durable data

The app has one aggregate — the active workout — so a single cubit is the source of truth for loading, every mutation, persistence and error handling.



Cubits, not full Blocs

Explicit states — `loading`, `ready`, `empty`, `failure` — keep every UI situation intentional. Event modeling would add ceremony without value at this size.



One versioned document

The whole workout is stored as a single JSON snapshot (`schemaVersion`, `updatedAt`, exercises with order + completion) via `shared_preferences`.



Atomic saves

Whole-document writes after every successful mutation — a save either fully succeeds or the previous document stays intact.



Never overwrite corruption

Corrupt saved data is never silently replaced; the user gets an explicit recovery choice (try again / start empty).



Single commit path

Every mutation — including reset and start-empty — flows through one `_commitSession` method that stamps the version + timestamp and persists. Save-failure and retry behavior is uniform for free.



Celebration, once

A one-shot flag fires the completion celebration only on the transition to 100%, and never repeats after a restart — pinned by unit tests.

Dependencies, and why

A deliberately small set of well-understood packages. Every dependency earns its place.

PACKAGE	ROLE IN TRAININBLUE
flutter_bloc	Cubit state management; one source of truth with testable mutations.
equatable	Value equality for states and models → predictable rebuilds.
shared_preferences	Tiny, reliable key-value store for the single JSON snapshot.
flutter_screenutil	Responsive sizing against the 390×844 design canvas.
cupertino_icons	Default iOS-style icon set.
flutter_launcher_icons (dev)	Generates Android/iOS launcher icons from the brand mark.
flutter_lints (dev)	Recommended lint set enforced by <code>flutter analyze</code> .

Platform & requirements



Flutter 3.41.x

Stable channel with Dart 3.11 — verify with `flutter --version`.



iOS & Android

Runs on a simulator, emulator, or a physical device. No network required.



Manrope font

Bundled weights 400–800 for a consistent, crisp type system.

Behavior pinned by tests

`dart format`, `flutter analyze` (zero issues) and `flutter test` (38 tests) were run after each significant step — not only at the end.

Unit tests

- ☐ Progress: empty / partial / complete / rounding
- ☐ JSON round-trips + invalid-data rejection
- ☐ Every cubit mutation incl. persistence
- ☐ Save-failure retry & recovery paths
- ☐ Celebration one-shot; corrupt-data protection

Widget tests

- ☐ Onboarding navigation with persisted completion
- ☐ Complete / reopen updates progress + persists
- ☐ Detail flow: set tracker counts, timer runs to zero
- ☐ Delete-from-detail returns home
- ☐ Empty state shows a safe 0%



Deterministic by construction

Fakes replace device storage (`FakeWorkoutRepository` , `SharedPreferences.setMockInitialValues`); an injectable clock and ID generator keep every test stable and repeatable.

38

Tests passing

0

Analyzer issues

≤150

Lines per file

Run from a clean checkout

Two commands to a running app. No environment variables, no accounts, no services to stand up.

1

Fetch dependencies

```
flutter pub get
```

2

Launch on your device or simulator

```
flutter run
```

Useful commands

TASK	COMMAND
Run the app	<code>flutter run</code>
Format	<code>dart format lib test</code>
Analyze	<code>flutter analyze</code>
Tests	<code>flutter test</code>
Regenerate launcher icons	<code>dart run flutter_launcher_icons</code>

**TrainInBlue**

Build. Train. Resume. — an offline-first workout builder that keeps a single session ready for you, every time you come back.

Version 1.0.0 · iOS & Android · Product Documentation